

Module 6: INSERT Statement

The **INSERT** statement is used to add new records into a database table. Every application, website, dashboard, and business system relies on INSERT operations to store data.



Introduction

Creating a table only defines the structure.

To actually store information, we need to insert data into the table.

Examples:

- Creating a new customer account
- Registering a student
- Adding a product to inventory
- Recording an employee's information

All these actions use the SQL **INSERT** statement.



Learning Objectives

After completing this module, you will be able to:

- ✓ Insert single records
 - ✓ Insert multiple records
 - ✓ Insert data into specific columns
 - ✓ Use NULL values correctly
 - ✓ Insert data from another table
 - ✓ Understand common INSERT errors
 - ✓ Follow data insertion best practices
-



Table of Contents

1. What is INSERT?
2. INSERT Syntax
3. Insert Single Record
4. Insert Multiple Records

5. Insert into Specific Columns
6. NULL Values
7. DEFAULT Values
8. INSERT INTO SELECT
9. Identity Columns
10. Common Errors
11. Best Practices
12. Summary
13. Practice Questions

1 What is INSERT?

The **INSERT** statement is a DML (Data Manipulation Language) command used to add records into a table.

Example

Table:

StudentID	StudentName	Age
101	Kunal	21

To add a new student:

```
INSERT INTO Students
VALUES
(102, 'Rahul', 22);
```

Result

StudentID	StudentName	Age
101	Kunal	21
102	Rahul	22

2 INSERT Syntax

Basic syntax:

```
INSERT INTO TableName
VALUES
```

```
(  
    Value1,  
    Value2,  
    Value3  
);
```

Example

```
INSERT INTO Employees  
VALUES  
(  
    101,  
    'Kunal',  
    45000  
);
```

Important Rule

Values must match:

- Column order
 - Data type
-

Example Table

```
CREATE TABLE Employees  
(  
    EmployeeID INT,  
    EmployeeName VARCHAR(100),  
    Salary DECIMAL(10,2)  
);
```

Insert Single Record

Adding one row at a time.

Example

```
INSERT INTO Employees
VALUES
(
    101,
    'Kunal',
    45000
);
```

Result

EmployeeID	EmployeeName	Salary
101	Kunal	45000

Another Example

```
INSERT INTO Employees
VALUES
(
    102,
    'Rahul',
    55000
);
```

Viewing Data

```
SELECT *
FROM Employees;
```



Insert Multiple Records

SQL allows inserting multiple rows in one statement.

Syntax

```
INSERT INTO TableName
VALUES
```

```
(Row1),  
(Row2),  
(Row3);
```

Example

```
INSERT INTO Employees  
VALUES  
(101, 'Kunal', 45000),  
  
(102, 'Rahul', 50000),  
  
(103, 'Aman', 55000),  
  
(104, 'Priya', 60000);
```

Result

EmployeeID	EmployeeName	Salary
101	Kunal	45000
102	Rahul	50000
103	Aman	55000
104	Priya	60000

Benefits

- ✓ Faster execution
- ✓ Cleaner code
- ✓ Fewer database calls

5 Insert into Specific Columns

Recommended approach.

Instead of relying on column order, specify column names.

Syntax

```
INSERT INTO TableName
(
    Column1,
    Column2
)
VALUES
(
    Value1,
    Value2
);
```

Example

Table:

```
CREATE TABLE Employees
(
    EmployeeID INT,
    EmployeeName VARCHAR(100),
    Salary DECIMAL(10,2),
    City VARCHAR(50)
);
```

Insert only selected columns:

```
INSERT INTO Employees
(
    EmployeeID,
    EmployeeName
)
VALUES
(
    101,
    'Kunal'
);
```

Result

EmployeeID	EmployeeName	Salary	City
101	Kunal	NULL	NULL

Why This Method Is Better

If table structure changes later:

```
ALTER TABLE Employees  
ADD Email VARCHAR(100);
```

Old INSERT statements still work.

6 NULL Values

NULL means:

Unknown
Missing
Not Available

Example

```
INSERT INTO Employees  
VALUES  
(  
    101,  
    'Kunal',  
    NULL  
);
```

Result

EmployeeID	EmployeeName	Salary
101	Kunal	NULL

Important

NULL is NOT:

✗ 0

✗ Blank Space

✗ Empty String

Example

```
SELECT *  
FROM Employees  
WHERE Salary IS NULL;
```

Checking Non-NULL

```
SELECT *  
FROM Employees  
WHERE Salary IS NOT NULL;
```

7 DEFAULT Values

If a column has a DEFAULT constraint, SQL inserts the default value automatically.

Example Table

```
CREATE TABLE Employees  
(  
    EmployeeID INT,  
    EmployeeName VARCHAR(100),  
    Country VARCHAR(50)  
    DEFAULT 'India'  
);
```

Insert

```
INSERT INTO Employees  
(
```



```
    EmployeeID,  
    EmployeeName  
  )  
VALUES  
(  
  101,  
  'Kunal'  
);
```

Result

EmployeeID	EmployeeName	Country
101	Kunal	India

Using DEFAULT Keyword

```
INSERT INTO Employees  
VALUES  
(  
  102,  
  'Rahul',  
  DEFAULT  
);
```

8 INSERT INTO SELECT

Copies data from one table to another.

Syntax

```
INSERT INTO TargetTable  
SELECT *  
FROM SourceTable;
```

Example

Employees Table

EmployeeID	Name
101	Kunal
102	Rahul

Backup Table

```
CREATE TABLE EmployeesBackup
(  
    EmployeeID INT,  
    Name VARCHAR(100)  
);
```

Copy Data

```
INSERT INTO EmployeesBackup  
SELECT *  
FROM Employees;
```

Result

EmployeesBackup now contains all records.

Insert Specific Columns

```
INSERT INTO EmployeesBackup  
(  
    EmployeeID,  
    Name  
)  
SELECT  
    EmployeeID,  
    Name  
FROM Employees;
```

Real-World Uses

- Data Migration

- Data Backup
- Data Warehousing
- Reporting Tables



Identity Columns

Identity columns generate values automatically.

Example

```
CREATE TABLE Employees
(
    EmployeeID INT IDENTITY(1,1),
    EmployeeName VARCHAR(100)
);
```

Meaning

```
IDENTITY(Start, Increment)
```

```
IDENTITY(1,1)
```

Produces:

```
1
2
3
4
5
```

Automatically.

Insert

```
INSERT INTO Employees
(
```

```
EmployeeName
)
VALUES
(
    'Kunal'
);
```

Result:

EmployeeID	EmployeeName
1	Kunal

Why Use Identity?

- ✓ Automatic IDs
- ✓ Prevent duplicates
- ✓ Simplifies inserts

10 Common INSERT Errors

Wrong Number of Values

Table:

```
CREATE TABLE Students
(
    StudentID INT,
    StudentName VARCHAR(100)
);
```

Bad:

```
INSERT INTO Students
VALUES
(
    101
);
```

Error:

```
Column count mismatch
```

Wrong Data Type

Bad:

```
INSERT INTO Students
VALUES
(
    'ABC',
    'Kunal'
);
```

Error:

```
Cannot convert VARCHAR to INT
```

Primary Key Violation

Table:

```
StudentID PRIMARY KEY
```

Bad:

```
INSERT INTO Students
VALUES
(
    101,
    'Kunal'
);

INSERT INTO Students
VALUES
```

```
(  
    101,  
    'Rahul'  
);
```

Error:

Duplicate key value

NOT NULL Violation

Bad:

```
INSERT INTO Students  
VALUES  
(  
    NULL,  
    'Kunal'  
);
```

Error:

Cannot insert NULL

Best Practices

Always Specify Columns

Preferred:

```
INSERT INTO Employees  
(  
    EmployeeID,  
    EmployeeName  
)  
VALUES  
(
```

```
    101,  
    'Kunal'  
);
```

Use Correct Data Types

Match values with column types.

Use Multiple Inserts

Good:

```
INSERT INTO Employees  
VALUES  
(101, 'Kunal'),  
  
(102, 'Rahul'),  
  
(103, 'Aman');
```

Use Identity Columns

For auto-generated IDs.

Validate Data Before Inserting

Check:

- Required fields
 - Data types
 - Business rules
-

Test Inserts in Development First

Avoid inserting incorrect production data.

Real-World Example

```
CREATE TABLE Employees  
(  
    EmployeeID INT IDENTITY(1,1)
```

```
PRIMARY KEY,  
  
EmployeeName NVARCHAR(100)  
NOT NULL,  
  
Email VARCHAR(150)  
UNIQUE,  
  
Salary DECIMAL(10,2)  
CHECK(Salary > 0),  
  
Country VARCHAR(50)  
DEFAULT 'India'  
);
```

Insert:

```
INSERT INTO Employees  
(  
    EmployeeName,  
    Email,  
    Salary  
)  
VALUES  
(  
    'Kunal Yadav',  
    'kunal@email.com',  
    45000  
)
```



Summary

In this module, you learned:

- ✓ INSERT INTO
- ✓ Single Row Insert
- ✓ Multiple Row Insert
- ✓ Specific Column Insert
- ✓ NULL Values
- ✓ DEFAULT Values
- ✓ INSERT INTO SELECT
- ✓ Identity Columns

- ✓ Common INSERT Errors
- ✓ Data Insertion Best Practices

Practice Questions

Theory

1. What is the INSERT statement?
 2. Why should column names be specified?
 3. What is NULL?
 4. Difference between NULL and 0?
 5. What is DEFAULT?
 6. What is an Identity column?
 7. What is INSERT INTO SELECT?
 8. Why use Identity columns?
 9. What causes a Primary Key violation?
 10. What are common INSERT errors?
-

Practical Exercises

Task 1

Create:

Students

Insert:

- 5 student records
-

Task 2

Create:

Employees

with:

- Identity column
- Default country

Insert:

- 10 employee records
-

Task 3

Create:

```
EmployeesBackup
```

Copy data using:

```
INSERT INTO SELECT
```

Task 4

Insert records with:

- NULL values
- DEFAULT values

Observe results.

Challenge Project

Create a mini College Database.

Tables:

```
Students  
Courses  
Enrollments
```

Insert:

- 20 students
- 5 courses
- 30 enrollment records

Maintain all constraints correctly.



Next Module

→ **Module 7: SELECT Statement**

Topics Covered:

- SELECT Syntax
- Selecting Specific Columns
- SELECT *
- Aliases
- DISTINCT
- TOP Clause
- Expressions
- Calculated Columns
- Query Execution Basics